Raster-Simulation

Joern Ploennigs · AI4SC



> The whole is greater than the sum of its parts.
>
> — Aristotle

Rasterproblemklasse

Was ist ein Rasterproblem?

Viele Umweltphänomene sind *kontinuierliche Felder*: Höhe, Temperatur, Bodenfeuchte existieren überall im Raum.

Konkrete Frage: Wie würdet ihr die Temperatur in Berlin speichern, wenn ihr nicht nur einzelne Messstationen, sondern die ganze Stadt beschreiben wollt?

Wie in der vorherigen Sitzung zu Geometrieproblemen gezeigt: Vektordaten beschreiben Grenzen und Objekte, Raster beschreiben flächige Felder mit einem Wert pro Zelle.

Um sie zu berechnen, *diskretisiert* man das Feld zu einem Gitter — einem **Raster**.

- *Zelle*: kleinste Einheit, speichert einen Mittelwert
- *Auflösung*: Kantenlänge einer Zelle (z. B. 25 m, 1 km)
- *Auflösung ist eine Modellentscheidung*: mehr Detail = mehr Rechenzeit

Rasterproblem = kontinuierliches Feld diskretisiert auf ein Gitter

> **i** Definition: Raster
>
> Ein *Raster* teilt einen räumlichen Bereich in ein regelmäßiges Gitter gleichgroßer Zellen. Jede Zelle erhält genau einen Wert.

Ein Raster entsteht, wenn ein kontinuierliches räumliches Phänomen in gleichgroße rechteckige Zellen eingeteilt wird. Die Wahl der Auflösung (Zellgröße) ist eine bewusste Modellierungsentscheidung: Feinere Gitter erfassen mehr räumliches Detail, erfordern aber mehr Speicher und Rechenzeit. Für globale Klimamodelle werden Zellgrößen von 1–25 km verwendet; für Hochwassermodelle in Städten reichen 1–5 m.

Die Verbindung zur vorherigen Sitzung ist zentral: Während Vektordaten Punkte, Linien und Polygone für klar abgegrenzte Objekte speichern, beschreiben Raster flächige Größen wie Höhe oder Temperatur. Beide Datenmodelle lösen also unterschiedliche Problemklassen.

Auflösung: Modellentscheidung

Feine Auflösung (1–25 m)

- Gebäude, Straßen sichtbar
- LiDAR-Befliegung, Drohnen
- Hochwassermodellierung
- Hoher Speicher- und Rechenaufwand

Grobe Auflösung (1–25 km)

- Globale Klimamodelle
- Niederschlagsradardaten
- Historische Klimaanalysen
- Schnelle Verarbeitung weltweit

Faustregel

Wählt die gröbste Auflösung, die noch die geforderte räumliche Genauigkeit liefert.

Aufgabe	Typische Zellgröße
Stadtplanung	1–5 m
Einzugsgebiet	10–50 m
Niederschlag (Region)	1 km
Klimamodell (Global)	25 km

Umwelttechnische Rasterbeispiele

Rasterbeispiele in der Umwelttechnik

Digitales Geländemodell

- Höhe der Erdoberfläche
- Auflösung: 1–90 m
- Ableitung: Hangneigung, Exposition, Einzugsgebiet
- Format: GeoTIFF

Niederschlag und Klima

- Radardaten (stündlich)
- Klimagitter (monatlich)
- Auflösung: 1–25 km
- Format: NetCDF, GeoTIFF

Landbedeckung und Luft

- Satelliten-Klassifikation: Wald / Acker / Siedlung
- Luftqualität: NO_2, $PM_{2.5}$
- Messstationen + Interpolation
- Auflösung: 10 m–10 km

Rasterdaten sind in der Umwelttechnik allgegenwärtig. Digitale Geländemodelle (DGM) bilden die Grundlage für Hochwassermodelle und Einzugsgebietsanalysen. Niederschlagsraster aus Radardaten oder Klimamodellen ermöglichen die Abschätzung von Starkregen-Ereignissen. Landbedeckungsklassifikationen aus Satellitendaten geben Aufschluss über Versiegelung und Verdunstungsverhalten. Luftqualitätsraster kombinieren Messstationen mit meteorologischen Modellen.

Digitales Geländemodell (DGM)

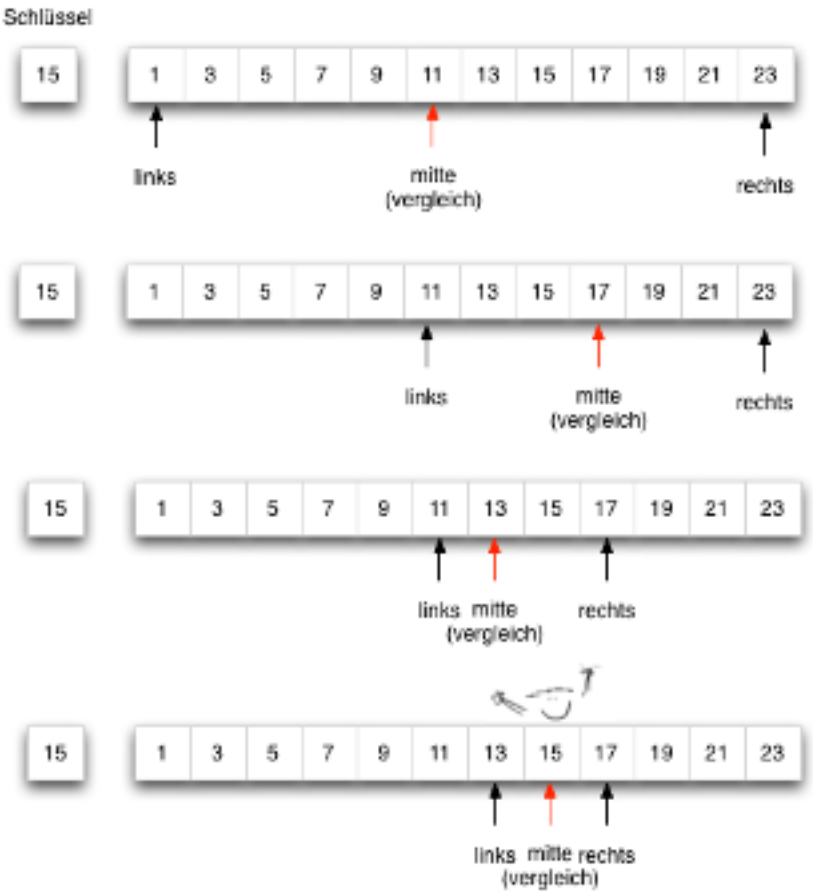
Das *Digitale Geländemodell* (DGM) ist das Standardraster in der Umwelttechnik — alle anderen Rasteranalysen bauen oft darauf auf.

Direkte Größen:

- Höhe über NN (Meter)
- NoData: Wasser, Ausland

Ableitbare Größen:

- *Hangneigung* (Slope): $\partial z/\partial x$, $\partial z/\partial y$
- *Exposition* (Aspect): Himmelsrichtung der Neigung
- *Einzugsgebiet*: alle Zellen, die in einen Punkt entwässern
- *Überflutungszone*: Zellen unter einem Schwellenhöhe



Midjourney: Digital elevation model hillshade visualization over a forested valley

Rasterdatenstruktur

Das 2D-Array als Raster

Ein Raster ist eine *2D-NumPy-Matrix* — Zeilen entsprechen Nord-Süd-Streifen, Spalten Ost-West-Streifen.

Pflicht-Metadaten:

Attribut	Bedeutung	Beispiel
`shape`	Zeilen × Spalten	`(200, 150)`
Zellgröße	Meter pro Zelle	`25 m`
Ausdehnung	Bounding Box	`xmin…ymax`
CRS	Koordinatensystem	`EPSG:25832`
NoData	Fehlender Wert	`-9999`

Datenstruktur

```
import numpy as np

# 5×5-Höhenraster (Meter über NN)
terrain = np.array([
    [120, 115, 110, 105, 100],
    [125, 118, 112, 108, 102],
    [130, 122, 115, 110, 105],
    [128, 120, 113, 108, 103],
    [125, 118, 110, 105, 100],
])

cell_size = 25   # Meter je Zelle
nodata    = -9999

print(terrain.shape)   # (5, 5)
print(terrain.dtype)   # int64
print(terrain.min())   # 100
```

Die minimale Datenstruktur für ein Raster ist ein NumPy-Array mit zugehörigen Metadaten: Zellgröße (wie viele Meter entspricht eine Zelle?), Ausdehnung (welchen geografischen Bereich deckt das Raster ab?), Koordinatensystem (CRS) und NoData-Wert (welcher Zahlenwert steht für fehlende Messungen?). Diese Metadaten sind nicht im Array selbst enthalten — sie müssen separat verwaltet werden. In Geo-Bibliotheken wie `rasterio` sind sie Teil der Datei (GeoTIFF), in einfachen Analysen werden sie als Variablen mitgeführt.

NoData-Werte: Fehlende Messungen

NoData markiert Zellen ohne gültigen Messwert:

- Wolken verdecken Satellitenbild
- Wasserflächen in Höhenmodellen
- Außerhalb des Messgebiets
- Defekter Sensor

Konventionen:

Konvention	Anwendung
`-9999`	Int-Raster, häufigster Wert
`np.nan`	Float-Raster
`255`	8-bit-Klassifikation

⚠ Achtung

NoData-Werte müssen **vor** allen Berechnungen maskiert werden — sonst verfälschen `-9999` die Mittelwerte und Summen.

```
nodata = -9999

# Gültige Zellen auswählen
valid_mask = terrain != nodata
valid_cells = terrain[valid_mask]

mean_height = valid_cells.mean()
print(f"Mittlere Höhe: {mean_height:.1f} m")

# Float-Variante mit np.nan
terrain_f = terrain.astype(float)
terrain_f[terrain_f == nodata] = np.nan

mean_nan = np.nanmean(terrain_f)
print(f"Nanmittel: {mean_nan:.1f} m")
```

Rasteroperationen

Reklassifizierung: Schwellenwert zur Maske

Reklassifizierung ordnet Zellen anhand ihres Wertes einer neuen Klasse zu. Der einfachste Fall: ein Schwellenwert erzeugt eine **binäre Maske**.

```
#| echo: true
import numpy as np

# Höhenraster (von vorhin)
terrain = np.array([
    [120, 115, 110, 105, 100],
    [125, 118, 112, 108, 102],
    [130, 122, 115, 110, 105],
    [128, 120, 113, 108, 103],
    [125, 118, 110, 105, 100],
])

# Überflutungsmaske: Höhe < 110 m
flood_mask = terrain < 110

gefaehrdet = np.sum(flood_mask)
```

```
anteil = gefaehrdet / terrain.size
print(f"Gefährdete Zellen: {gefaehrdet} ({anteil:.0%})")
```

Rasteroperation

> **i** Definition: Reklassifizierung
>
> *Reklassifizierung* ordnet jeder Rasterzelle eine neue Klasse zu — basierend auf dem ursprünglichen Wert.
>
> - Schwellenwert ⟶ binäre Maske (`True`/`False`)
> - Wertebereich ⟶ Hangneigungsklassen (flach/mittel/steil)
> - Lookup-Tabelle ⟶ Landnutzungsaggregation

Pattern C: Was gibt `flood_mask.dtype` zurück? Was ändert sich, wenn ihr den Schwellenwert auf 115 setzt?

Overlay: Zwei Raster verknüpfen

Overlay kombiniert zwei Raster mit gleicher Form, Zellgröße und Ausdehnung über eine elementweise Operation.

Beispiel: Risikoindex = Wahrscheinlichkeit × Schadenspotenzial

```
#| echo: true
# Überflutungswahrscheinlichkeit (0–1)
prob = np.array([
    [0.0, 0.0, 0.1, 0.8, 0.9],
    [0.0, 0.0, 0.1, 0.7, 0.8],
    [0.0, 0.0, 0.0, 0.2, 0.6],
])

# Schadenspotenzial (0–1)
damage = np.array([
    [0.3, 0.4, 0.5, 0.8, 0.2],
    [0.3, 0.4, 0.6, 0.9, 0.3],
    [0.2, 0.3, 0.5, 0.7, 0.4],
])

# Element-weises Overlay
risk = prob * damage
print(risk.max())   # Höchstes Risiko
```

⚠ Ausdehnung prüfen!

Overlay setzt identische Form voraus. KI prüft das **nicht automatisch**.

```
# Vor jedem Overlay prüfen:
assert prob.shape == damage.shape, (
    f"Form stimmt nicht: "
    f"{prob.shape} vs {damage.shape}"
)
```

Wenn Ausdehnung oder Zellgröße abweicht, muss vorher *resampled* werden.

Zonenstatistik

Zonenstatistik berechnet einen Aggregatwert eines Rasters innerhalb definierter Zonen (z. B. Teileinzugsgebiete, Verwaltungsgrenzen).

```
#| echo: true
# Zonenraster (0 = keine Zone)
zones = np.array([
    [1, 1, 1, 2, 2],
    [1, 1, 1, 2, 2],
    [1, 1, 0, 2, 2],
    [0, 0, 0, 2, 2],
    [0, 0, 0, 0, 0],
])

for zone_id in [1, 2]:
    mask = zones == zone_id
    mean_h = terrain[mask].mean()
    print(f"Zone {zone_id}: ø {mean_h:.1f} m")
```

Ausgabe:

```
Zone 1: ø 119.4 m
Zone 2: ø 105.4 m
```

i Zonenstatistik

Kombiniert:

- *Werteraster* — was wird gemessen? (Höhe, Niederschlag)
- *Zonenraster* — welche Gebiete? (Einzugsgebiete, Gemeinden)
- *Aggregationsfunktion* — wie wird zusammengefasst? (`mean`, `sum`, `max`)

In der Praxis liefert `rasterstats` oder `zonal_stats()` diese Funktion.

Pattern C: Warum wird `terrain[mask]` als 1D-Array zurückgegeben? Was passiert bei Zone 0?

numpy-Demo: Geländeanalyse

KI-generierter Code — Geländeraster laden

Pattern C — KI-Code lesen

```
#| echo: true
import numpy as np
import matplotlib.pyplot as plt

# Synthetisches Höhenprofil von Nord nach Süd
np.random.seed(42)
height_profile = np.linspace(50, 200, 20)
print("Profil (erste 5 Werte):", height_profile[:5])

# Profil auf alle 20 Spalten ausbreiten
base = height_profile.reshape(20, 1) * np.ones((1, 20))
print("Basisraster-Form:", base.shape)

# Zufällige lokale Abweichungen hinzufügen
noise = np.random.normal(0, 8, (20, 20))
terrain20 = (base + noise).astype(int)

cell_size = 25   # Meter je Zelle
nodata    = -9999

print(f"Form:      {terrain20.shape}")
print(f"Min / Max: {terrain20.min()} / {terrain20.max()} m")
print(f"Mittelwert: {terrain20.mean():.1f} m")
```

Lest den Code bevor ihr ihn ausführt: Was gibt `terrain20.shape` zurück? Welchen Wertebereich erwarten wir für Min/Max? Was bewirkt `np.random.seed(42)`?

KI-generierter Code — Hangneigung berechnen

```
#| echo: true
# Hangneigung mit np.gradient
# Gibt zwei Arrays zurück: dy (N–S), dx (O–W)
dy, dx = np.gradient(terrain20,
                     cell_size, cell_size)
slope = np.sqrt(dx**2 + dy**2)

# Einheiten umrechnen
slope_pct = slope * 100          # Prozent
slope_deg = np.degrees(
    np.arctan(slope))             # Grad

print(f"Max. Neigung: {slope_pct.max():.1f} %")
print(f"ø   Neigung:  {slope_pct.mean():.1f} %")
```

Fragen:

- Welche Einheit hat `slope` direkt nach `np.gradient`?
- Warum `np.sqrt(dx**2 + dy**2)`?
- Was bedeutet eine Neigung von 100 %?

i Hangneigung

`np.gradient` berechnet numerische Ableitung:

$$\text{slope} = \sqrt{\left(\frac{\partial z}{\partial x}\right)^2 + \left(\frac{\partial z}{\partial y}\right)^2}$$

- Einheit nach Gradient: m/m (dimensionslos)
- × 100 ⟶ Prozent
- `np.degrees(np.arctan(…))` ⟶ Grad

Für `cell_size = 25 m`: eine Höhendifferenz von 25 m auf einer Zelle = 100 % Neigung = 45°.

KI-generierter Code — Maske und Visualisierung

```
#| echo: true
#| fig-height: 3.5
threshold = 100   # Meter – Überflutungsgrenze

flood_mask20 = terrain20 < threshold

fig, axes = plt.subplots(1, 3, figsize=(12, 3.5))
```

```
im0 = axes[0].imshow(terrain20, cmap="terrain")
axes[0].set_title("Geländehöhe (m)")
plt.colorbar(im0, ax=axes[0])

im1 = axes[1].imshow(slope_pct, cmap="Reds")
axes[1].set_title("Hangneigung (%)")
plt.colorbar(im1, ax=axes[1])

im2 = axes[2].imshow(flood_mask20, cmap="Blues")
axes[2].set_title(f"Risiko (< {threshold} m)")

plt.tight_layout()
plt.show()
```

Die drei Teilgrafiken zeigen das Geländehöhenraster mit Terrain-Colormap, die Hangneigung in Prozent (rote Farbreihe: steilere Bereiche), sowie die binäre Überflutungsmaske. `imshow` interpretiert den Boolean-Array direkt: `True` (= 1) ⟶ blaue Zellen, `False` (= 0) ⟶ helle Zellen. Das Layout `plt.subplots(1, 3)` erzeugt drei nebeneinander stehende Achsen.

KI-Fallstricke und Prompt-Muster

Was KI vergisst: Metadaten-Fallstricke

KI rechnet korrekt …

- Element-weise Operationen
- `mean`, `sum`, `max`
- Schwellenwert-Masken
- Visualisierungen mit `imshow`

… aber prüft nicht:

- Zellgröße — Ergebnis in falscher Einheit
- Ausdehnung — Overlay auf falsches Gebiet
- NoData — Fehlwerte in Berechnungen
- CRS — Koordinatensysteme passen nicht

⚠️ Typischer KI-Fehler

```
# KI generiert (ohne vollständigen Prompt):
def fraction_above(grid, threshold):
    return np.mean(grid > threshold)

# Problem: Mit nodata = -9999 und threshold = 0.5
# ist -9999 < 0.5 → False → scheinbar korrekt.
# Aber: NoData-Zellen gehen trotzdem in den Nenner ein.
# Bei Mittelwerten dominiert -9999 sogar als extremer Ausreißer.
```

Testet **immer** mit einem Gitter, das einen NoData-Wert enthält.

Fehlerbeispiel: NoData in der Berechnung

```
#| echo: true
import numpy as np

# Bodenfeuchtigkeit mit NoData-Zellen
moisture = np.array([
    [0.3, 0.7, -9999],
    [0.5, 0.8, -9999],
    [0.2, 0.6,  0.9],
])
nodata = -9999

# ❌ KI-Code ohne NoData-Berücksichtigung
def fraction_above_bad(grid, t):
    return np.mean(grid > t)

bad = fraction_above_bad(moisture, 0.5)
print(f"Falsch:  {bad:.2f}")
# -9999 > 0.5 → False → zählt als 0,
# und vergrößert damit den Nenner trotz ungültiger Zellen.

# ✅ Mit NoData-Filterung
valid = moisture[moisture != nodata]
ok = np.mean(valid > 0.5)
print(f"Richtig: {ok:.2f}")
```

Immer zuerst `grid[grid != nodata]` filtern

Agentic workflow — Verifikation:

1. Prüft, ob KI `nodata` in der Funktion berücksichtigt

2. Testet mit einem Gitter, das `-9999` enthält
3. Präzisiert den Prompt: *„NoData-Wert ist –9999 — schließe ihn vor allen Berechnungen aus"*

KI-Prompt-Muster für Rasterprobleme

Allgemeines Muster

```
Gegeben ein 2D-NumPy-Array, das [Phänomen]
repräsentiert — Zellgröße [X m],
NoData-Wert [-9999]:

1. [Operation beschreiben]
2. Schließe NoData-Zellen aus
3. Gib [Ausgabetyp] zurück
4. Teste auf kleinem synthetischen Array
```

Vollständiges Beispiel

```
Gegeben ein 2D-NumPy-Array `moisture`
mit shape (rows, cols), Zellgröße 50 m,
NoData-Wert -9999:

Schreibe `fraction_above(grid, nodata, t)`,
die den Anteil gültiger Zellen > t zurückgibt.
Rückgabe: float (0.0–1.0).
Teste mit einem 3×3-Array mit 2 NoData-Zellen.
```

> **i** Muster-Elemente
>
> Vollständiges Prompt enthält immer:
>
> - **Phänomen** — was misst das Raster?
> - **Zellgröße** — damit Einheiten stimmen
> - **NoData-Wert** — damit Filterung passiert
> - **Ausgabetyp** — float, int, bool, Array?
> - **Testfall** — KI soll Verifikation einbauen
>
> Je präziser die Metadaten im Prompt, desto seltener muss man iterieren.

Prompt-Ergebnis: KI-Code lesen und verifizieren

```
#| echo: true
# KI-generierter Code (nach vollständigem Prompt)
def fraction_above(grid, nodata, threshold):
    """Anteil gültiger Zellen über threshold."""
    valid = grid[grid != nodata]
```

```
    if len(valid) == 0:
        return float("nan")
    return float(np.mean(valid > threshold))

# Integrierter Testfall (von KI generiert)
test = np.array([[0.3, 0.7, -9999],
                 [0.5, 0.8, -9999],
                 [0.2, 0.6,  0.9]])

result = fraction_above(test, nodata=-9999, threshold=0.5)
expected = 3 / 7   # 3 Zellen > 0.5 von 7 gültigen

assert abs(result - expected) < 1e-9, f"Fehler: {result}"
print(f"✓  fraction_above = {result:.3f}")
```

Prüft den Code: Stimmt `expected = 3 / 7`? Was passiert, wenn alle Zellen NoData sind? Ist das sicher gehandhabt?

Verifikations-Checkliste

Checkliste: Rasteranalyse verifizieren

Form und Ausdehnung

- Shape stimmt mit Ausdehnung überein
- Zellgröße ist dokumentiert und korrekt
- Extent von Eingabe und Ergebnis gleich
- CRS ist bekannt und konsistent
- Overlay-Raster haben identische Form

Werte und Einheiten

- NoData vor Berechnungen maskiert
- Einheit des Ergebnisses klar (m, %, 0–1)
- Min/Max im plausiblen Bereich
- Ausgabetyp korrekt (float, int, bool)

Kleiner Testfall als Pflicht

```
# Synthetisches 3×3-Raster mit bekanntem Ergebnis
test = np.array([
    [1.0,  2.0, -9999],
    [3.0,  4.0,  5.0],
    [6.0, -9999, 7.0],
])
# Gültige Zellen: 1,2,3,4,5,6,7 → 7 Stück
# Davon > 3.0: 4,5,6,7 → 4 Stück → 4/7 ≈ 0.571
```

```
result = fraction_above(test, -9999, 3.0)
assert abs(result - 4/7) < 1e-9
print("✓ Test bestanden")
```

Merksatz

Testet **immer** zuerst auf einem kleinen synthetischen Raster mit bekanntem Ergebnis — bevor ihr echte Geodaten verwendet.

Agentischer Workflow: Rasterproblem

Agentic Cycle

Agentischer Workflow: 6-schrittiger Prozess zur KI-generierten Rasteranalyse.

1. **Erkennen** — ist es ein Rasterproblem? Kontinuierliches Feld? Grid-Darstellung geeignet?
2. **Spezifizieren** — Metadaten vollständig in den Prompt: Shape, Zellgröße, NoData-Wert, Einheit, CRS
3. **Generieren** — Ausgabetyp und NoData-Anforderung explizit nennen; Testfall anfordern
4. **Verstehen** — Code lesen, nicht nur ausführen: Wo wird NoData maskiert? Stimmt die Einheit?
5. **Verifizieren** — Checkliste abarbeiten: Shape, NoData, Einheit, Plausibilität, Testfall
6. **Iterieren** — Prompt mit fehlenden Metadaten präzisieren

Verbindungen

Vorgänger:

→ `10b_Geometrieprobleme` (S10): Vektordaten und Rasterdaten sind komplementäre Raumdarstellungen — wann welche wählen?

Nachfolger:

→ `11b_UIDesign` (S11): Rasterergebnisse (Masken, Risikokarten) werden in einem interaktiven Dashboard visualisiert.

KI erzeugt korrektes Array-Rechnen — aber du musst die Metadaten spezifizieren und das Ergebnis gegen bekannte Werte prüfen.

Referenzen

Bibliographie